Advanced Workshop

Tiny Tapeout — From Synthesis to Silicon Layout

Northwestern IEEE · Open Silicon 2026

Date: Friday April 10, 2026 **Room:** Tech L440 **Time:** 5–9 PM

Duration: ≈3 hours of curriculum

Prerequisites: Beginner session, or comfortable with Verilog and GitHub Actions.

Tools: VGA Playground · OSS CAD Suite · Yosys · iic-osic-tools (Docker) · XSCHEM · NGSPICE · Magic

PDK: SkyWater sky130A (130 nm)

Goal: Understand the full stack — from synthesized digital to hand-crafted analog — and submit both types of design to Tiny Tapeout.

Contents

1 What This Lab Is / Is Not **2**

2 The Session Arc **2**

3 Part 1: VGA Playground — Verilog in Practice **2**

4 Part 2: OSS CAD Suite — Synthesis Under the Hood **3**
4.1 Install . . . 3
4.2 OSS CAD Suite vs. Quartus / Synplify . . . 3
4.3 The Local Workflow . . . 3
4.4 What `stat` Tells You . . . 4
4.5 From Tiny Tapeout to Production . . . 4

5 Part 3: The Mixed-Signal Reveal **5**
5.1 What This Means for sky130 . . . 5

6 Part 4: The Analog Flow — XSCHEM and NGSPICE **6**
6.1 Setup: iic-osic-tools . . . 6
6.1.1 macOS . . . 6
6.1.2 Linux . . . 7
6.1.3 Windows . . . 7
6.2 Create a Project . . . 8
6.3 Draw the Schematic in XSCHEM . . . 8
6.3.1 Place Transistors . . . 8
6.3.2 Add Pins . . . 8
6.3.3 Wire the Inverter . . . 8
6.3.4 Set Transistor Parameters . . . 8
6.4 Create a Symbol . . . 9
6.5 Simulate with NGSPICE . . . 9
6.5.1 Build the Testbench . . . 9
6.5.2 Configure Component Values . . . 9
6.5.3 Add Simulation Commands . . . 10
6.5.4 Run . . . 10
6.5.5 Add Measurements . . . 10

7 Part 5: Physical Layout in Magic **10**
7.1 Key sky130 Layers . . . 11
7.2 Magic Hotkeys Reference . . . 11
7.3 Layout Architecture: CMOS Inverter . . . 11
7.4 Draw the Layout — Step by Step . . . 12
7.5 DRC . . . 13
7.6 LVS . . . 13

8 Part 6: Parasitic Extraction and Post-Layout Simulation **14**
8.1 Run Post-Layout Simulation . . . 14
8.2 Compare Results . . . 14

9 Part 7: GDS Submission to Tiny Tapeout **14**

9.1 Export from Magic 15
9.2 Use the sky130 Analog Template 15

10 Key Deadlines **15**

11 XSCHEM Shortcuts Reference **16**

12 Further Resources **16**

Appendix A: Full Inverter Tutorial Reference **18**

1 What This Lab Is / Is Not

This lab is an introduction to **open-source EDA tools**, not a course in ECE. By your junior or senior year you have already learned the concepts — transistors, logic, circuit analysis. We are here to show you the tools that let you do this stuff in open source.

If you have no background yet: you can keep up. Everything is guided. If you do: this is where you can contribute. These tools are not up to par with the billion-dollar commercial EDA industry *yet* — and that gap is a research opportunity.

2 The Session Arc

In the beginner session you went from transistor → logic gate → Wokwi → Verilog → GDS. That arc is powerful, but it hides something important. Today we pull back the curtain:

VGA Playground → OSS CAD Suite → The Mixed-Signal Reveal → XSCHEM → NGSPICE → Magic → Extraction → GDS Submission

By the end of today you will understand why the inverter you drew in SiliWiz and the inverter Yosys inserts during synthesis are the same thing — and why that matters.

3 Part 1: VGA Playground — Verilog in Practice

Link: vga-playground.com

> *In the beginner session, Wokwi was your breadboard. Now you are writing real code. VGA Playground is your serial monitor with live output — Verilog on the left, a 640×480 screen on the right. This is the jump from Arduino blocks to C++.*

1. Open vga-playground.com. Pick **Stripes, Rings, Logo, or Checkers** (the only presets that fit in one TT tile).
2. Make one visible change. For Stripes, find:

```
wire [9:0] moving_x = pin_x + counter;
```

Change `+` to `-`. Stripes reverse. One token, immediate visual result.

3. When you are happy with your design, copy the Verilog source.
4. Submit it to TT using the IHP template:

- Fork **github.com/TinyTapeout/ttihp-verilog-template**
- Paste your code into `src/project.v`, rename the module to `tt_um_YOURGITHUBNAME_vga`
- Update `info.yaml` and `docs/info.md`
- Add `cocotb.pass_test()` to `test/test.py`
- Add `hvsync_generator.v` to `PROJECT_SOURCES` in `test/Makefile`

- Commit → wait for `gds` and `docs` green → submit at app.tinytapeout.com

4 Part 2: OSS CAD Suite — Synthesis Under the Hood

Download: github.com/YosysHQ/oss-cad-suite-build/releases

> *This is when you stop using the Arduino IDE and install `gcc`. OSS CAD Suite gives you the exact same Yosys locally. You see every step the compiler takes and understand what it actually produces.*

4.1 Install

Download the file for your platform:

OS	File to download
Linux (x64)	`oss-cad-suite-linux-x64-*.tgz`
Linux (ARM64)	`oss-cad-suite-linux-arm64-*.tgz`
Mac (Apple Silicon)	`oss-cad-suite-darwin-arm64-*.tgz`
Mac (Intel)	`oss-cad-suite-darwin-x64-*.tgz`
Windows (x64)	`oss-cad-suite-windows-x64-*.exe`

```
# Linux / macOS --- extract and activate
tar -xf oss-cad-suite-*.tgz
source oss-cad-suite/environment    # every new terminal

# Windows --- run the .exe installer, then:
oss-cad-suite\environment.bat

# Verify
yosys --version && iverilog -V
```

4.2 OSS CAD Suite vs. Quartus / Synplify

If you have used Quartus or Vivado, this will feel different. There is no GUI, no project wizard, no schematic editor.

Tool	What it does	Quartus equivalent
`yosys`	RTL synthesis (Verilog → gates)	Analysis & Synthesis
`nextpnr`	Place-and-route (gates → layout)	Fitter
`iverilog`	Verilog simulation	ModelSim / Questa
`gtkwave`	Waveform viewer (VCD files)	SignalTap / ModelSim waves
`sby`	Formal verification	—

Languages: Verilog (native), SystemVerilog (via `yosys-slang` plugin), VHDL (via `ghdl` plugin).

Editing: Any text editor. VS Code + *Lushay Code* extension for syntax highlighting and lint.

4.3 The Local Workflow

```
# Run from the workshop repo (NOT oss-cad-suite, it has its own examples
   / folder)
# Simulate
iverilog -o sim examples/tb.v examples/project.v
```

```
vvp sim

# View waveforms (expand tb > dut, append signals,
# Shift+Alt+F to zoom to fit)
gtkwave tb.vcd

# Synthesize + view gate-level schematic
yosys -p "read_verilog examples/project.v; \
          synth -top tt_um_yourname_counter; stat; show"
```

4.4 What stat Tells You

After synthesis, Yosys prints a cell inventory. For a 4-bit counter you will see something like:

```
Number of wires:                12
Number of cells:                10
  sky130_fd_sc_hd__dfxtp_1      4    <-- 4 flip-flops (one per bit)
  sky130_fd_sc_hd__inv_1        2    <-- inverters
  sky130_fd_sc_hd__nand2_1      4    <-- NAND gates for the adder logic
```

Every cell name starts with `sky130_fd_sc_hd__` — the sky130 **standard cell library**. These are pre-drawn, pre-characterized transistor-level circuits. Yosys does not invent new transistor arrangements; it selects from this fixed menu and wires them together.

How to know what is synthesizable: if Yosys errors or warns, it cannot build it. `#delays`, division `/`, `initial` blocks, and large memories will either fail or be silently ignored.

How to know what will fail TT CI: run `yosys` locally with the same `synth` command. If it passes locally, it will pass in GitHub Actions.

What you will NOT get locally: post-layout timing analysis. TT's CI runs OpenSTA for static timing analysis after place-and-route. OSS CAD Suite gives you logic correctness, not timing sign-off.

> **Note**
>
> **This is the bridge:** the `inv_1` cell Yosys just inserted is an XSCHEM schematic, an NGSPICE simulation, and a Magic layout that someone built by hand — exactly the flow we are about to do in Parts 4–6.

4.5 From Tiny Tapeout to Production

What changes when you go from TT to a real product at TI, a startup, or a foundry?

	Tiny Tapeout	Production ASIC
Synthesis	Yosys (open)	Synopsys Design Compiler / Cadence Genus
Place & route	OpenLane (open)	Cadence Innovus / Synopsys ICC2
Timing	OpenSTA in CI	Synopsys PrimeTime (signoff-grade STA)
Simulation	iverilog	Synopsys VCS / Cadence Xcelium
Schematic	XSCHEM (open)	Cadence Virtuoso
Layout	Magic (open)	Cadence Virtuoso Layout
PDK	sky130 (open, 130 nm)	TSMC / GlobalFoundries / Samsung (NDA)
Cost	∼$150 / tile	$100K–$1M+ mask set

The workflow is the same: RTL → synthesis → P&R → DRC/LVS → tapeout. The tools change. The concepts do not.

What is portable: your Verilog RTL, your testbenches, your design skills.

What is not portable: your layout. You re-harden with the target PDK. A sky130 GDS will not run on a TSMC 28 nm process — but the Verilog that generated it will.

5 Part 3: The Mixed-Signal Reveal

Here is the thing synthesis tutorials do not say out loud:

Synthesis can only produce circuits that switch between two stable voltage levels. Anything that does not fit that description cannot be synthesized.

Circuit	Why it cannot be synthesized
Ring oscillator / VCO	Output frequency depends on transistor physics, not logic
PLL	Feedback across analog and digital domains
Bandgap voltage ref	Output set by physical constants, not logic equations
Comparator	Operates on continuous voltages, not discrete bits
ADC / DAC	Converts between continuous and discrete — inherently analog
ESD protection cells	Voltage clamps, diodes — purely physical
I/O buffers	Must drive real off-chip loads with controlled slew rates
Crystal osc interface	Analog feedback loop

Now look at any real chip — phone SoC, microcontroller, sensor IC. Every one contains at least a power-on reset (analog), I/O buffers (analog), and usually a clock source (PLL or oscillator — analog). Most contain ADCs or DACs.

Every chip that contains any of the above is a mixed-signal ASIC. That includes almost every chip ever manufactured.

5.1 What This Means for sky130

Sky130 is designed for both digital and analog work. It has two separate sets of devices:

`sky130_fd_sc_hd` — Standard cell library (digital)
Yosys picks from here. Pre-built, pre-verified, never touched by hand.

`sky130_fd_pr` — Foundry primitives (analog)
Raw transistors (`nfet_01v8`, `pfet_01v8`), resistors, capacitors, diodes. You place these by hand in XSCHEM and simulate them in NGSPICE.

When you designed the inverter in SiliWiz, you were working in the `sky130_fd_pr` world — drawing actual transistors. When Yosys inserts an `inv_1` cell from `sky130_fd_sc_hd`, it is using a cell that was built in exactly the same way, just already done for you.

> *In Arduino terms, you have been using pre-built libraries. Now you are writing the library from scratch. The advanced flow is the `sky130_fd_pr` flow — you are hand-building the components that Yosys picks off the shelf.*

6 Part 4: The Analog Flow — XSCHEM and NGSPICE

We will build a CMOS inverter (`my_inverter`) from scratch — the same gate SiliWiz showed you, now in a full professional EDA flow.

6.1 Setup: iic-osic-tools

The iic-osic-tools Docker container provides XSCHEM, NGSPICE, Magic, KLayout, and Netgen in a single pre-configured environment. **Docker Desktop must be installed and running before you begin.**

6.1.1 macOS

```
# 1. Install Docker Desktop (requires macOS Sonoma or newer)
brew install --cask docker
open -a Docker
# Wait for the whale icon in the menu bar to stop animating.

# 2. Install XQuartz (once)
brew install --cask xquartz

# 3. Log out of macOS and log back in (required for XQuartz to register)

# 4. Open XQuartz from /Applications/Utilities/
#    XQuartz -> Settings (or Cmd+,) -> Security tab
#    -> check "Allow connections from network clients"
#    Quit XQuartz (Cmd+Q) and reopen it.

# 5. Allow Docker to use your display
xhost +local:docker
# If xhost fails, try:  xhost +si:localuser:$USER

# 6. Clone the tools repo
git clone https://github.com/hpretl/iic-osic-tools.git
cd iic-osic-tools

# 7. Start the container (~12 GB download on first run)
./start_x.sh
```

macOS older than Sonoma (pre-14.0)

Docker Desktop requires macOS Sonoma or newer. On older macOS, use **colima** instead:

```
brew install colima docker
colima start
export DOCKER_HOST=unix://$HOME/.colima/default/docker.sock
```

Then continue from step 2 above. Use `./start_vnc.sh` and open `http://localhost:80` if X11 gives trouble.

6.1.2 Linux

```
# 1. Start Docker if not running
sudo systemctl start docker

# 2. Allow Docker to use your display
xhost +local:docker
# If xhost fails, try:  xhost +si:localuser:$USER

# 3. Clone the tools repo
git clone https://github.com/hpretl/iic-osic-tools.git
cd iic-osic-tools

# 4. Start the container (~12 GB download on first run)
./start_x.sh
```

Note

If you get "permission denied" on Docker commands, add yourself to the docker group: `sudo usermod -aG docker $USER` then log out and back in.

6.1.3 Windows

```
# 1. Install Docker Desktop with the WSL 2 backend
#    https://docs.docker.com/desktop/install/windows-install/

# 2. Open a WSL terminal (Ubuntu recommended)
wsl

# 3. Clone the tools repo
git clone https://github.com/hpretl/iic-osic-tools.git
cd iic-osic-tools

# 4. Start the container in VNC mode (no X11 needed)
./start_vnc.sh

# 5. Open a browser and go to:
#    http://localhost:80
```

Note

On Windows, use `start_vnc.sh` instead of `start_x.sh`. This runs a VNC server inside the container and exposes it as a web page at `http://localhost:80` — no X server install needed.

All subsequent commands run **inside the container** unless stated otherwise.

Note

Files saved to `/foss/designs` inside the container persist at `~/eda/designs` on your host machine.

6.2 Create a Project

```
mkdir -p /foss/designs/my_inverter
cd /foss/designs/my_inverter
xschem
```

6.3 Draw the Schematic in XSCHEM

XSCHEM is a schematic capture tool. You build your circuit by placing component symbols and wiring them together.

6.3.1 Place Transistors

1. Press `Insert` to open the symbol chooser.
2. Navigate to `/foss/pdks/sky130A/libs.tech/xschem/sky130_fd_pr/`
3. Select `nfet_01v8.sym` — place it on the canvas (1.8 V NMOS).
4. Press `Insert` again, select `pfet_01v8.sym` — place it above the NMOS.

6.3.2 Add Pins

1. `Insert` → `devices/ipin.sym` → place left, press `Q`, set `lab=in`.
2. `Insert` → `devices/opin.sym` → place right, `Q`, `lab=out`.
3. `Insert` → `devices/iopin.sym` → place two: top (`lab=vdd`), bottom (`lab=gnd`).

6.3.3 Wire the Inverter

- PMOS gate + NMOS gate → `in`
- PMOS drain + NMOS drain → `out`
- PMOS source → `vdd`; NMOS source → `gnd`
- PMOS bulk (B) → `vdd`; NMOS bulk (B) → `gnd`

Press `W` to draw wires. Click pin endpoints to connect. A filled dot appears where wires join.

6.3.4 Set Transistor Parameters

Select each transistor, press `Q`:

- NMOS: `W=1`, `L=0.15` (minimum gate length for sky130)
- PMOS: `W=2`, `L=0.15` (2× width to compensate lower hole mobility)

Note

Why W=2 for PMOS? Hole mobility is roughly half electron mobility. Doubling PMOS width equalises rise and fall times — the output switches symmetrically.

Save as `/foss/designs/my_inverter/my_inverter.sch` (`Ctrl+Shift+S`).

6.4 Create a Symbol

The symbol is a black-box view of your schematic for use in testbenches and higher-level designs.

1. With `my_inverter.sch` open: **Symbol → Make symbol from schematic**.
2. XSCHEM generates `my_inverter.sym` in the same directory.
3. Save with `Ctrl+S`.

Note

If you add or remove pins later, re-run **Symbol → Make symbol from schematic** to update it.

6.5 Simulate with NGSPICE

Simulation uses a **testbench schematic** — a separate `.sch` that instantiates your inverter symbol and adds stimulus sources.

6.5.1 Build the Testbench

1. **File → New Schematic**.
2. `Insert` → navigate to `/foss/designs/my_inverter/`, place `my_inverter.sym`.
3. `Insert` → place `devices/vsource.sym` — this is `Vvdd` (power supply).
4. `Insert` → place another `vsource.sym` — this drives the `in` pin.
5. `Insert` → place `devices/capa.sym` — load capacitor at `out`.
6. `Insert` → place `devices/gnd.sym` — ground symbol.
7. `Insert` → navigate to `/foss/pdks/sky130A/libs.tech/xschem/sky130_fd_pr/corner.sym`, place it, press `Q`, set `corner=tt` (typical-typical).

Warning

If you skip the corner symbol, NGSPICE will fail with "model not found."

6.5.2 Configure Component Values

- Select `Vvdd`, press `Q`, set `value=1.8`.
- Select the input vsource, press `Q`, set value to:

```
PULSE(0 1.8 1n 100p 100p 5n 10n)
```

(low=0 V, high=1.8 V, delay=1 ns, rise=100 ps, fall=100 ps, pulse-width=5 ns, period=10 ns)

- Select the capacitor, press `Q`, set `value=10f`.

6.5.3 Add Simulation Commands

Place `devices/code_shown.sym`, press `Q`, enter:

```
.temp 27
.control
tran 0.1n 30n
write tb_my_inverter.raw
.endc
```

Place `devices/lab_wire.sym` labels on the `in`, `out`, `vdd` nets.

Simulation → **Set netlist dir** → select `/foss/designs/my_inverter/`.

Save as `/foss/designs/my_inverter/tb_my_inverter.sch` (`Ctrl+Shift+S`).

6.5.4 Run

1. Click **Netlist**, then **Simulate**. A terminal shows NGSPICE output.
2. Click **Waves** to open GAW. **File** → **Open** → select `tb_my_inverter.raw`.
3. Drag `v(in)` and `v(out)` into the waveform panel. You should see clean inversion.

6.5.5 Add Measurements

Place another `code_shown.sym` and add:

```
.meas tran tphl TRIG v(in) VAL=0.9 RISE=1 TARG v(out) VAL=0.9 FALL=1
.meas tran tplh TRIG v(in) VAL=0.9 FALL=1 TARG v(out) VAL=0.9 RISE=1
.meas tran trise TRIG v(out) VAL='1.8*0.1' RISE=1 TARG v(out) VAL
   ='1.8*0.9' RISE=1
.meas tran tfall TRIG v(out) VAL='1.8*0.9' FALL=1 TARG v(out) VAL
   ='1.8*0.1' FALL=1
```

Re-run **Netlist** → **Simulate**. Propagation delay and rise/fall times print in the console.

7 Part 5: Physical Layout in Magic

Magic VLSI is the layout tool for custom analog cells. You draw the physical shapes that correspond to your schematic — the same layers you saw in SiliWiz, at full precision.

```
cd /foss/designs/my_inverter
magic -rcfile $PDK_ROOT/sky130A/libs.tech/magic/sky130A.magicrc &
```

7.1 Key sky130 Layers

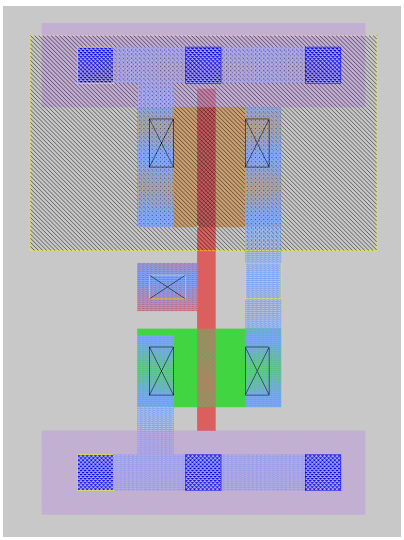
Layer	Purpose
`nwell`	N-well for PMOS
`ndiff / pdiff`	Active diffusion (source/drain)
`poly`	Polysilicon gate
`ncontact / pcontact`	Contacts to li1
`li1`	Local interconnect (metal 0)
`m1`	Metal 1

7.2 Magic Hotkeys Reference

Key	Action
`Space`	Cycle through tools (box, wiring, pick)
`Left-click`	Set one corner of box
`Right-click`	Set opposite corner of box
`Middle-click`	Paint layer from palette into box
`p`	Paint selected layer into box (same as middle-click)
`e / ^D`	Erase material in box / delete selected
`s`	Select area under cursor
`a`	Select everything inside box
`I`	Select cell instance
`M`	Move selected
`C`	Copy selected
`b`	Report box coordinates and size
`x`	Expand cell (show internal layers)
`z / Z`	Zoom in / zoom out
`:`	Open Magic command line (Tcl console)

7.3 Layout Architecture: CMOS Inverter

The standard CMOS inverter layout follows a vertical stack. Use this reference image and layer guide while you draw:



Reference: sky130 inverter layout (Matt Venn, github.com/mattvenn/magic-inverter). Yellow outline = nwell, green = ndiff, red = poly gate, cyan = li1/metal, blue = m1 contacts, grey bars = power rails.

Reading the image top to bottom:

- **Top:** m1 VDD rail. PMOS source contacts connect up to it.
- **Upper half:** PMOS (pdiff inside nwell). Poly gate crosses horizontally.
- **Middle:** Poly gate runs vertically connecting both gates (`in`). li1 wire connects PMOS drain to NMOS drain (`out`).
- **Lower half:** NMOS (ndiff in substrate). Same poly gate crosses it.
- **Bottom:** m1 GND rail. NMOS source contacts connect down to it.

7.4 Draw the Layout — Step by Step

Option A: Use the PDK device generator (recommended for beginners):

1. In Magic: **Device 1** menu → select **nmos** (sky130_fd_pr___nfet_01v8), set W=1, L=0.15 → **Create** → **Close**. Click away from the device, then repeat for **pmos** (sky130_fd_pr___pfet_01v8), W=2, L=0.15.
2. Use `I` to select each cell, `M` to move. Stack PMOS above NMOS, gates aligned vertically.
3. Draw an **nwell** rectangle enclosing the entire PMOS device (with margin).

Option B: Draw from scratch (for the adventurous):

1. Draw an **nwell** rectangle in the upper half for the PMOS.
2. Draw **pdiff** inside the nwell (PMOS active region).
3. Draw **ndiff** in the lower half, outside the nwell (NMOS active region).
4. Draw a **poly** stripe crossing both diff regions — this is the shared gate. Width: 0.15 μm (sky130 minimum gate length).
5. Add **pcontact** squares inside the pdiff source/drain regions.
6. Add **ncontact** squares inside the ndiff source/drain regions.

Then wire it up (both options):

1. Route **li1** wires:
   - Connect PMOS drain contact and NMOS drain contact together → `out` net. Draw a box overlapping both drain contacts, then `paint li` (or middle-click li1 in palette).
   - PMOS source contact → VDD rail (top)
   - NMOS source contact → GND rail (bottom)
   - Poly gate → `in` label (left side)
2. Add **m1** horizontal bars for VDD (top) and GND (bottom) power rails. Use `paint m1` after boxing the rail area.
3. Label ports: box over each net, then in the console:

```
label in FreeSans 100 0 0 0 c poly
label out FreeSans 100 0 0 0 c li
label VDD FreeSans 100 0 0 0 c m1
label VSS FreeSans 100 0 0 0 c m1
port make
```

Save in the Magic console: `save my_inverter`

7.5 DRC

Design Rule Check verifies your layout meets the foundry's spacing and width rules.

```
# In the Magic console
drc check
drc why
```

Common violations:

- Poly too narrow (minimum 0.15 μm)
- Diff-to-diff spacing too small (minimum 0.27 μm)
- Missing nwell tap or pwell tap (tie the well to supply)

Iterate until `drc why` returns no errors.

7.6 LVS

Layout vs. Schematic confirms the layout is electrically identical to the schematic.

1. In XSCHEM with `my_inverter.sch` open, click **Netlist** → generates `my_inverter.spice`.
2. Run Netgen:

```
netgen -batch lvs \
  "my_inverter.spice my_inverter" \
  "$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/spice/sky130_fd_sc_hd.
     spice my_inverter" \
  $PDK_ROOT/sky130A/libs.tech/netgen/sky130A_setup.tcl \
  my_inverter_lvs.out
```

Check the result at the bottom of `my_inverter_lvs.out`:

- *Circuits match uniquely.* — pass, move on
- *Netlists do not match.* — debug: check pin labels, source/drain orientation, missing connections

Note

Common LVS failures: **Missing net** (unconnected wire), **Swapped ports** (source/drain swapped), **Extra device** (stray poly/diff overlap creating an unintended transistor).

8 Part 6: Parasitic Extraction and Post-Layout Simulation

The schematic simulation assumed ideal wires with zero resistance and capacitance. Real metal wires have both. Parasitic extraction (PEX) pulls these values from the physical layout and annotates them into a new SPICE netlist.

```
magic -rcfile $PDK_ROOT/sky130A/libs.tech/magic/sky130A.magicrc << 'EOF'
load my_inverter
flatten my_inverter_flat
load my_inverter_flat
extract do local
extract all
ext2spice lvs
ext2spice cthresh 0
ext2spice extresist on
ext2spice
EOF
```

This produces `my_inverter_flat.spice` — the layout netlist with parasitic `R` and `C` elements on every net.

8.1 Run Post-Layout Simulation

1. Open `tb_my_inverter.sch` in XSCHEM.
2. Add a `code_shown.sym` with:

```
.include /foss/designs/my_inverter/my_inverter_flat.spice
```

3. Temporarily disable the instance of `my_inverter.sym` (select it, press `Q`, add `spice_ignore=true`).
4. **Netlist** → **Simulate** → **Waves**.

8.2 Compare Results

Parameter	Schematic sim	Post-layout sim
Rise time	baseline	slower (wire capacitance)
Fall time	baseline	slower
Prop delay	baseline	slower

The post-layout values will be slightly worse. This is expected — it is the accurate result. If the degradation is larger than expected, find and reduce the parasitic hotspots (long thin wires, tight vias).

9 Part 7: GDS Submission to Tiny Tapeout

Tiny Tapeout accepts analog designs as pre-built GDS + LEF files that get dropped into the chip-level floorplan.

9.1 Export from Magic

```
# In the Magic console
gds write my_inverter.gds
lef write my_inverter.lef
```

9.2 Use the sky130 Analog Template

1. Fork **github.com/TinyTapeout/tt10-analog-template** (or the current shuttle's analog template).
2. Drop `my_inverter.gds` into `gds/` and `my_inverter.lef` into `lef/`.
3. Update `info.yaml`:
   - `analog: true`
   - `top_module`: your cell name
   - `title`, `author`, `description`
4. Update `docs/info.md` — describe pins, operating conditions, how to test.
5. Commit → watch the `gds` action (pre-check DRC + 3D viewer).
6. Once green: **app.tinytapeout.com** → **Create project** → **Submit a new revision**.

Note

Analog tiles go through a manual review step in addition to the automated checks. Allow extra time before the deadline.

10 Key Deadlines

Date	Milestone
April 10	Advanced workshop (today)
April 17	Beginner workshop
May 1	IEEE form submission
May 5	Token distribution
May 9	**Final submission deadline**

11 XSCHEM Shortcuts Reference

Key	Action
`Insert`	Place symbol
`W`	Draw wire
`Q`	Edit properties
`M`	Move
`C`	Copy
`Del`	Delete
`R`	Rotate while placing
`F`	Flip while placing
`Ctrl+Z`	Undo
`Ctrl+S`	Save
`Ctrl+Shift+S`	Save As
`E`	Descend into symbol
`Ctrl+E`	Return to parent
Scroll	Zoom
Middle-drag	Pan

You started with a breadboard (Wokwi), moved to real code (Verilog), opened the compiler (Yosys), and then built the library component by hand (XSCHEM → NGSPICE → Magic → GDS). A standard cell is just an Arduino library someone else wrote and tested — now you know how to write one yourself.

12 Further Resources

Keep learning after the workshop. These are the best free resources for open-source chip design.

Courses and Guides

1. **Zero to ASIC Course** (Matt Venn) — full digital and analog tapeout courses using open-source tools.
   https://www.zerotoasiccourse.com
2. **Tiny Tapeout Documentation** — official guides for digital, analog, and mixed-signal submissions.
   https://tinytapeout.com/docs
3. **Tiny Tapeout Analog Specs** — pin mapping, tile sizes, and submission requirements.
   https://tinytapeout.com/specs/analog
4. **SSCS Open-Source Ecosystem** (IEEE Solid-State Circuits Society) — curated analog IC design resources.
   https://sscs-ose.github.io/analog
5. **Skywater PDK Documentation** — design rules, device models, and layer definitions.
   https://skywater-pdk.readthedocs.io

Tutorials and Blog Posts

6. **Doing Magic in Magic VLSI** (Cornell C2S2) — step-by-step layout tutorial with DRC walkthrough.
   https://c2s2.engineering.cornell.edu/blogposts/SP24/Magic.html

7. **Magic Inverter Layout + Simulation** (Matt Venn) — complete sky130 inverter with Makefile.
   https://github.com/mattvenn/magic-inverter
8. **sky130 Inverter Layout Tutorial** (Gonzaga University) — PDF walkthrough of Magic with sky130.
   http://web02.gonzaga.edu/faculty/talarico/vlsi/13_magic_inverter_sky130.pdf
9. **Open-Source Analog Design Examples** — xschem, ngspice, and gaw setup and usage.
   https://github.com/eescottie/opensrc_analog
10. **Chip Tutorials — sky130 PDK** (VLSIDA) — concise reference for PDK structure and tool setup.
    https://vlsida.github.io/chip-tutorials/sky130.html

YouTube Channels

11. **Psychogenic Technologies** — deep dives on open-source EDA, Magic, and chip design workflows.
    https://www.youtube.com/@PsychogenicTechnologies
12. **Matt Venn / Zero to ASIC** — tapeout walkthroughs, tool demos, and interviews.
    https://www.youtube.com/@mattvenn

Tools and Infrastructure

13. **iic-osic-tools** (Harald Pretl, JKU) — the Docker container used in this workshop.
    https://github.com/hpretl/iic-osic-tools
14. **Tiny Tapeout Analog VM** — pre-built VirtualBox VM with all tools and sky130 PDK.
    https://github.com/TinyTapeout/analog-virtualbox-vm-sky130a

Appendix A: Full Inverter Tutorial Reference

This appendix is a condensed reference for the complete XSCHEM → NGSPICE → Magic → PEX flow. The main session document covers the same steps with more explanation. Use this as a quick lookup during the hands-on portion.

Initialization

```
# Mac only (once)
brew install --cask xquartz
# Log out and back in; enable "Allow connections from network clients"
   in XQuartz Preferences

# All platforms
xhost +local:docker
git clone https://github.com/hpretl/iic-osic-tools.git
cd iic-osic-tools
./start_x.sh          # ~12 GB first pull; opens container GUI window
```

All subsequent commands run **inside the container** unless stated otherwise. Files in `/foss/designs` persist at `~/eda/designs` on the host.

Project Setup

```
mkdir -p /foss/designs/my_inverter
cd /foss/designs/my_inverter
xschem
```

XSCHEM Schematic Shortcuts

Key	Action
`Insert`	Place symbol
`W`	Draw wire
`Q`	Edit properties
`M`	Move
`C`	Copy
`Del`	Delete
`R`	Rotate while placing
`F`	Flip while placing
`Ctrl+Z`	Undo
`Ctrl+S`	Save
`Ctrl+Shift+S`	Save As
`E`	Descend into symbol
`Ctrl+E`	Return to parent
Scroll	Zoom
Middle-drag	Pan

Schematic: Place Transistors

1. **Insert** → navigate to `/foss/pdks/sky130A/libs.tech/xschem/sky130_fd_pr/`

2. Select `nfet_01v8.sym` (NMOS) and place.
3. **Insert** again, select `pfet_01v8.sym` (PMOS), place above NMOS.

Schematic: Add Pins

1. **Insert** → `devices/ipin.sym` → place left, press `Q`, set `lab=in`.
2. **Insert** → `devices/opin.sym` → place right, `Q`, `lab=out`.
3. **Insert** → `devices/iopin.sym` → place top (`lab=vdd`) and bottom (`lab=gnd`).

Schematic: Wire the Inverter

- PMOS gate + NMOS gate → `in`
- PMOS drain + NMOS drain → `out`
- PMOS source → `vdd`; NMOS source → `gnd`
- PMOS bulk (B) → `vdd`; NMOS bulk (B) → `gnd`

Schematic: Transistor Parameters

Select each transistor, press `Q`:

- NMOS: `W=1`, `L=0.15` (minimum gate length for sky130)
- PMOS: `W=2`, `L=0.15` (2× width to compensate lower hole mobility)

Save as `/foss/designs/my_inverter/my_inverter.sch`.
Symbol → **Make symbol from schematic** to generate `my_inverter.sym`.

Testbench & Simulation

New schematic. Place: `my_inverter.sym`, `vsource.sym` (×2), `capa.sym`, `gnd.sym`, `corner.sym` (press `Q`, set `corner=tt`).

Component values:

- `Vvdd`: `value=1.8`
- Input vsource: `value=PULSE(0 1.8 1n 100p 100p 5n 10n)`
- Capacitor: `value=10f`

Place `code_shown.sym`, press `Q`, enter:

```
.temp 27
.control
tran 0.1n 30n
write tb_my_inverter.raw
.endc
```

Set netlist dir: **Simulation** → **Set netlist dir** → `/foss/designs/my_inverter/`.
Save as `tb_my_inverter.sch`. Click **Netlist** → **Simulate** → **Waves**.

Optional measurements (second `code_shown.sym`):

```
.meas tran tphl TRIG v(in) VAL=0.9 RISE=1 TARG v(out) VAL=0.9 FALL=1
.meas tran tplh TRIG v(in) VAL=0.9 FALL=1 TARG v(out) VAL=0.9 RISE=1
.meas tran trise TRIG v(out) VAL='1.8*0.1' RISE=1 TARG v(out) VAL
    ='1.8*0.9' RISE=1
.meas tran tfall TRIG v(out) VAL='1.8*0.9' FALL=1 TARG v(out) VAL
    ='1.8*0.1' FALL=1
```

Magic Layout

```
magic -rcfile $PDK_ROOT/sky130A/libs.tech/magic/sky130A.magicrc &
```

Key layers: `nwell`, `ndiff/pdiff`, `poly`, `ncontact/pcontact`, `li1`, `m1`.

Hotkeys: Space (cycle tools), left/right-click (set box corners), middle-click (paint from palette), `p` (paint), `e` (erase), `s` (select), `I` (select cell), `M` (move), `z/Z` (zoom in/out), `b` (box info), `x` (expand cell).

Recommended: use Device 1 menu to generate nfet_01v8 (W=1, L=0.15) and pfet_01v8 (W=2, L=0.15) as pre-validated cells. Stack PMOS above NMOS, gates aligned.

Layout stack (bottom to top): m1 GND rail → ncontact/ndiff (NMOS) → poly gate (shared, crossing both) → li1 drain-to-drain wire (`out`) → pdiff/pcontact inside nwell (PMOS) → m1 VDD rail. Label ports with `label <name>` then `port make`.

In Magic console: `save my_inverter`, `drc check`, `drc why`.

LVS

```
netgen -batch lvs \
  "my_inverter.spice my_inverter" \
  "$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/spice/sky130_fd_sc_hd.
     spice my_inverter" \
  $PDK_ROOT/sky130A/libs.tech/netgen/sky130A_setup.tcl \
  my_inverter_lvs.out
```

Check `my_inverter_lvs.out` for *Circuits match uniquely.*

Parasitic Extraction (PEX)

```
magic -rcfile $PDK_ROOT/sky130A/libs.tech/magic/sky130A.magicrc << 'EOF'
load my_inverter
flatten my_inverter_flat
load my_inverter_flat
extract do local
extract all
ext2spice lvs
ext2spice cthresh 0
ext2spice extresist on
ext2spice
EOF
```

Produces `my_inverter_flat.spice`. Use in testbench with `.include` to run post-layout simulation.

GDS Export & Submission

```
# In Magic console
gds write my_inverter.gds
lef write my_inverter.lef
```

Fork `github.com/TinyTapeout/tt10-analog-template`. Place `my_inverter.gds` in `gds/` and `my_inverter.lef` in `lef/`. Update `info.yaml` (`analog: true`, `top_module`, `title`, `author`, `description`). Commit → wait for `gds` and `docs` green → `app.tinytapeout.com` → Submit.